

Konwerter kodu Ada $\rightarrow$ C

Dokumentacja techniczna

Autorzy:	Natalia Pożniak Małgorzata Poskróbek
Język impl.:	Java 17
Generator parsera:	ANTLR4 4.13.1
Framework web:	Spring Boot 3.2.3
System budowania:	Maven

Spis treści

1	Opis projektu	2
2	Technologie	2
3	Architektura systemu	2
4	Opis klas	3
4.1	Application	3
4.2	Main	3
4.3	Compiler	3
4.4	CompilerController	3
4.5	AdaToCVisitor	4
4.5.1	Zarządzanie zakresami	4
4.5.2	System typów	4
4.5.3	Generowanie kodu	4
5	Obsługiwane konstrukcje języka Ada	4
6	Analiza semantyczna	5
7	Gramatyka	5
7.1	Lekser – AdaLexer.g4	5
7.2	Parser – AdaParser.g4	6
8	Interfejs webowy	6
9	Uruchomienie	6
9.1	Aplikacja webowa	6
9.2	Tryb CLI	7
10	Przykłady	7
10.1	Silnia (Ada)	7
10.2	Wygenerowany kod C	7
11	Tabela tokenów	7

1 Opis projektu

Projekt stanowi **transpiler** (source-to-source compiler) tłumaczący kod źródłowy napisany w języku **Ada** na ekwiwalentny kod w języku **C**. Udostępnia dwa tryby działania:

- **Aplikacja webowa** – interfejs przeglądarkowy oparty na Spring Boot, pozwalający wpisać kod Ady i natychmiast uzyskać wynik konwersji,
- **Tryb CLI** – uruchamiany bezpośrednio z wiersza poleceń; przetwarza wskazane pliki `.adb` i zapisuje wyniki do katalogu `out/`.

2 Technologie

Warstwa	Technologia	Wersja
Język implementacji	Java	17
Framework webowy	Spring Boot (spring-boot-starter-web)	3.2.3
Generator parsera	ANTLR4	4.13.1
System budowania	Apache Maven	—
Frontend	HTML + Bootstrap 5.3 + vanilla JS	—

Tabela 1: Stos technologiczny projektu

3 Architektura systemu

Pipeline przetwarzania kodu przebiega przez cztery etapy:

1. **Leksykalna analiza** – `AdaLexer` (generowany z `AdaLexer.g4`) zamienia strumień znaków na sekwencję tokenów.
2. **Analiza składniowa** – `AdaParser` (generowany z `AdaParser.g4`) buduje drzewo rozbioru (*ParseTree*).
3. **Analiza semantyczna i generowanie kodu** – `AdaToCVisitor` przechodzi drzewo metodą *Visitor* i produkuje kod C.
4. **Wyjście** – tekst kodu C zwracany do wywołującego (plik `.c` w trybie CLI, odpowiedź JSON w trybie web).

Listing 1: Schemat pipeline'u

```

Kod Ada (tekst)
  |
  v
AdaLexer          <- tokenizacja (AdaLexer.g4)
  |
  v
AdaParser         <- analiza skladniowa (AdaParser.g4)
  | ParseTree
  v
AdaToCVisitor     <- semantyka + generowanie kodu C
  |
  v

```

Kod C (tekst)

4 Opis klas

4.1 Application

Punkt startowy aplikacji webowej. Uruchamia kontener Spring Boot na porcie 8080.

Listing 2: Klasa Application

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

4.2 Main

Punkt startowy trybu CLI. Przyjmuje listę plików `.adb` jako argumenty, dla każdego wywołuje `Compiler.compileFile()` i zapisuje wynik do katalogu `out/`. Błędy syntaktyczne i semantyczne trafiają na `stderr`; program kończy się kodem 2 w przypadku niepowodzenia.

4.3 Compiler

Fasada enkapsulująca pipeline ANTLR4 dla trybu plikowego. Odpowiada za:

- wczytanie pliku (`CharStreams.fromFileName()`),
- konfigurację leksera i parsera z własnym `ErrorListener` rzucającym `ParseCancellationException` przy błędzie składni,
- uruchomienie `AdaToCVisitor`,
- udostępnienie listy błędów semantycznych przez `getSemanticErrors()` i `hasSemanticErrors()`.

4.4 CompilerController

Kontroler REST Spring Boot (`@RestController`).

- **Endpoint:** POST `/api/compile`
- **Wejście:** JSON `{"code": "..."}`
- **Wyjście (sukces):** `{"success": true, "cCode": "..."}`
- **Wyjście (błąd):** `{"success": false, "errors": [...]}`

Pipeline działa na łańcuchu znaków (`CharStreams.fromString()`), bez operacji wejścia-/wyjścia na dysku.

4.5 AdaToCVisitor

Główna klasa logiki transpilatora – rozszerza wygenerowaną klasę `AdaParserBaseVisitor<String>`. Każda metoda `visitXxx()` odpowiada regule gramatycznej i zwraca fragment kodu C jako `String`.

4.5.1 Zarządzanie zakresami

Klasa utrzymuje stos (`Deque`) dwóch równoległych struktur:

- `scopeTypes` – `Map<String, CType>` mapująca nazwę zmiennej na jej typ,
- `declaredVars` – `Set<String>` zmiennych zadeklarowanych w bieżącym zakresie.

Metody `pushScope()` / `popScope()` są wywoływane na wejściu i wyjściu z każdej procedury i funkcji.

4.5.2 System typów

Wewnętrzny enum `CType`:

Listing 3: Enum `CType`

```
private enum CType {
    INT("int"),
    DOUBLE("double"),
    BOOL("bool");
}
```

Mapowanie typów Ada → C:

Typ Ada	Typ C
Integer, Natural, Positive	int
Float	double
Boolean	bool

Tabela 2: Mapowanie typów

Metoda `inferType()` przechodzi rekurencyjnie przez węzły `expression` → `term` → `factor` i wyznacza typ wynikowy wyrażenia.

4.5.3 Generowanie kodu

Wcięcia zarządzane są zmienną `indentLevel` (4 spacje na poziom). Metoda `indent()` generuje odpowiedni łańcuch białych znaków.

5 Obsługiwane konstrukcje języka Ada

Konstrukcja Ada	Odpowiednik C
<code>procedure P is ... end P;</code>	<code>void P() { ... }</code>
<code>procedure Main is ...</code>	<code>int main() { ... return 0; }</code>

Konstrukcja Ada	Odpowiednik C
function F return T is ...	T F() { ... }
declare X := E;	deklaracja zmiennej lokalnej
X := E;	X = E;
if ... then ... elsif ... else ... end if;	if (...) { ... } else if { ... } else {
while ... loop ... end loop;	while (...) { ... }
for I in A .. B loop ...	for (int I = A; I <= B; I++) { }
return E;	return E;
Put_Line(E);	printf("%d\n", E) / %f
A(I), A(I,J)	A[I], A[I][J]
=, /=	==, !=

Tabela 3: Obsługiwane konstrukcje

6 Analiza semantyczna

Klasa `AdaToCVisitor` przeprowadza analizę semantyczną w trakcie przechodzenia drzewa. Wykryte błędy są zbierane na liście `semanticErrors` i zwracane po zakończeniu kompilacji.

Rodzaj błędu	Przykład wyzwalający
Użycie niezadeklarowanej zmiennej	<code>x := y + 1</code> (y nieznane)
Niezgodność typów w przypisaniu	przypisanie <code>double</code> do <code>int</code>
Operacja arytmetyczna na <code>bool</code>	<code>True + 1</code>
Dzielenie przez zero (statyczne)	<code>x := a / 0;</code>
Nieprawidłowy typ indeksu tablicy	<code>A(1.5)</code>
Indeks tablicy poza zakresem	<code>A(10)</code> przy rozmiarze 5
Pusta funkcja (brak <code>return</code>)	<code>function F return Integer is begin end F;</code>

Tabela 4: Wykrywane błędy semantyczne

7 Gramatyka

Gramatyka jest podzielona na dwa pliki ANTLR4.

7.1 Lekser – `AdaLexer.g4`

Lekser obsługuje słowa kluczowe *case-insensitive* (Ada jest językiem nierozróżniającym wielkości liter) za pomocą fragmentów literowych:

Listing 4: Fragment `AdaLexer.g4`

```
PROCEDURE : P R O C E D U R E ;
FUNCTION  : F U N C T I O N ;
...
FLOAT     : [0-9]+ '.' [0-9]+ ;
INTEGER   : [0-9]+ ;
IDENTIFIER : [a-zA-Z_] [a-zA-Z0-9_]* ;
COMMENT   : '--' ~[\r\n]* -> skip ;
```

```
WS      : [ \t\r\n]+ -> skip ;
fragment A:[aA]; fragment B:[bB]; ...
```

7.2 Parser – AdaParser.g4

Listing 5: Gramatyka (BNF – skrót)

```
program      ::= subprogram_list
subprogram_decl ::= procedure_decl | function_decl

procedure_decl ::= "procedure" IDENTIFIER "is"
                declaration_part?
                "begin"
                proc_statement_list?
                "end" IDENTIFIER ";"

function_decl  ::= "function" IDENTIFIER "return" IDENTIFIER "is"
                declaration_part?
                "begin"
                func_statement_list?
                "end" IDENTIFIER ";"

declaration_part ::= "declare" declaration+ "begin" | eps
declaration      ::= IDENTIFIER ":@" expression ";"

proc_statement   ::= assignment | if_statement | while_statement
                  | for_statement | put_line_statement
func_statement   ::= assignment | if_statement | while_statement
                  | for_statement | return_statement

condition        ::= expression relational_op expression
relational_op     ::= "=" | "/=" | "<" | ">" | "<=" | ">="
expression       ::= term | expression ("+" | "-") term
term              ::= factor | term ("*" | "/") factor
factor           ::= "-" factor | IDENTIFIER | array_access
                  | INTEGER | FLOAT | "(" expression ")"
```

8 Interfejs webowy

Frontend serwowany jest jako plik statyczny z `src/main/resources/static/index.html`.

- **Lewy panel:** pole tekstowe z kodem Ada (monospacefont, ciemne tło).
- **Przycisk *Konwertuj do C*:** wywołuje POST `/api/compile` przez Fetch API.
- **Prawy panel:** wygenerowany kod C (kolor niebieski) lub komunikat o błędzie.
- **Sekcja błędów semantycznych:** lista błędów wyświetlana na czerwono poniżej paneli.

9 Uruchomienie

9.1 Aplikacja webowa

```
mvn spring-boot:run
```

Aplikacja dostępna pod adresem <http://localhost:8080>.

9.2 Tryb CLI

```
mvn package -DskipTests
java -jar target/ada-to-c-1.0.jar examples/factorial.adb
```

Wynik zapisywany jest w katalogu `out/factorial.c`. W przypadku błędów program wypisuje komunikaty na `stderr` i kończy działanie z kodem 2.

10 Przykłady

10.1 Silnia (Ada)

Listing 6: examples/factorial.adb

```
procedure Main is
declare
  n := 5;
  result := 1;
begin
  for i in 1 .. n loop
    result := result * i;
  end loop;
  Put_Line(result);
end Main;
```

10.2 Wygenerowany kod C

Listing 7: out/factorial.c

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

int main() {
  int n = 5;
  int result = 1;
  for (int i = 1; i <= n; i++) {
    result = result * i;
  }
  printf("%d\n", result);
  return 0;
}
```

11 Tabela tokenów

Token	Wzorzec / Definicja	Opis
PROCEDURE	"procedure" (case-insensitive)	deklaracja procedury
FUNCTION	"function"	deklaracja funkcji
IS	"is"	poczatek sekcji deklaracji
BEGIN	"begin"	poczatek bloku instrukcji
END	"end"	koniec bloku
IF	"if"	instrukcja warunkowa
THEN	"then"	czesc warunku if
ELSE	"else"	alternatywa if
ELSIF	"elsif"	dodatkowy warunek if
WHILE	"while"	petla while
LOOP	"loop"	blok petli
FOR	"for"	petla for
IN	"in"	operator zakresu w for
RETURN	"return"	zwrot wartosci
DECLARE	"declare"	sekcja deklaracji lokalnych
ASSIGN	:=	przypisanie wartosci
RANGE	..	operator zakresu
PLUS	+	dodawanie
MINUS	-	odejmowanie
MUL	*	mnozenie
DIV	/	dzielenie
EQ	=	rownosc
NEQ	/=	nierownosc
LT	<	mniejsze
GT	>	wieksze
LE	<=	mniejsze lub rowne
GE	>=	wieksze lub rowne
SEMICOLON	;	koniec instrukcji
COMMA	,	separator
LPAREN	(	nawias otwierajacy
RPAREN	)	nawias zamykajacy
INTEGER	[0-9]+	liczba calkowita
FLOAT	[0-9]+.[0-9]+	liczba zmiennoprzecinkowa
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	nazwa zmiennej/procedury
COMMENT	-- ~[\r\n]*	komentarz (pomijany)
WS	[\t\r\n]+	biale znaki (pomijane)

Tabela 5: Tabela tokenow leksera